Production Test System — Proposal

Hardware Inside M17 (hw 1.5, scc3)

Component	Part	Role
MCU	STM32G031G8 (Cortex-M0+, 64 MHz)	Main controller
Motor driver	AT5833	Stepper driver, STEP/DIR interface from MCU; nFault fault line to MCU
Position sensors	3× analog Hall sensors	ADC inputs; read by firmware for closed-loop position feedback
RS485 transceiver	SN65HVD3082	230,400 baud differential bus
Supply sense	Resistor divider → ADC	Measures input voltage
Temperature	ADC (thermistor)	Board temperature monitoring
LEDs	Green (PC15), Red (PC14)	Status indicators

Rack Layout

- 6 shelves × 24 slots = **144 motors**
- Shelves paired into 3 independent RS485 sections (A = shelves 1–2, B = 3–4, C = 5–6)
- 3 USB-RS485 adapters, one per section
- Power constraint: **8 motors at full power simultaneously** per section

---

Parallelism Strategy

RS485 is half-duplex, but motor moves execute asynchronously. After sending a motion command, the motor runs it independently while the host addresses the next motor. This allows a **fan-out/gather** pattern:

1. Send motion command to motors 1–8 in rapid succession
2. Poll all 8 for completion while they run simultaneously
3. Collect results, move to next group of 8

Power constraint enforced by software: no more than 8 motors in high-power phases at once. All 3 bus sections run in parallel (3 threads), giving a total of up to 24 motors under power at once across the rack.

Device Detection & Building the Test Set

Before any testing, the operator builds a **test set** per bus — the fixed list of motors that the test phases will run against. Detection (`detect_devices`, cmd 20) is used *only* here. Once the test set is built, the system never detects again; every test phase addresses motors by their unique ID.

Per-bus detect controls (3 independent buses)

Each bus (A, B, C) has its own **Detect** button and its own counters. The three buses are fully independent — separate detect queues, separate test sets, separate lists on screen — so there is never any ambiguity about which bus a motor belongs to.

Pending-detections counter (per bus): - Each click of a bus's Detect button **increments** that bus's pending counter by 1. - A background worker runs one `detect_devices` pass per pending count; each completed pass **decrements** the counter by 1. - The operator can click multiple times to queue several passes (detection is probabilistic with collision-avoidance delays, so multiple passes improve the chance of finding every motor on a crowded bus). - The counter is shown live so the operator can see how many passes are still queued / running.

Test set (per bus): - Motors found across all passes accumulate into that bus's test set (union — a motor seen in any pass stays in the set). - The list is shown live on screen and updates as each pass completes. - A **count of motors in the set** is shown for each bus (e.g. "Bus A: 47 motors") so the operator can confirm all expected units were found before starting tests.

Clear buttons: - Each bus has its own **Clear** button that empties only that bus's test set. - A single **Clear All** button empties all three test sets at once. - Clearing affects only the in-memory test set — it never deletes anything from the database. (This is exactly the situation that later produces an orange highlight: an ID that is still in the DB but no longer in the current test set.)

New-vs-known colour coding (catching non-unique unique IDs)

The **main purpose** of this colouring is to surface the case where a unique ID is not actually unique — i.e. more than one physical motor carries the same ID. The colour does not by itself declare a fault; it gives the operator the information to judge, based on what they did (e.g. whether they

cleared the set or closed the program).

As each motor is detected, its colour is decided from two facts — whether its ID is already in **this bus's test set**, and whether its ID is already in the **database**:

In test set?	In DB?	Colour	Meaning
no	no	**green**	brand-new motor → added to DB (with first-detection time) + test set
yes	yes	**green**	already in this session's set → normal re-detection
no	yes	**orange**	ID is in the DB but not in this session's set → recorded before (prior session, or after a Clear), **or a second motor shares this "unique" ID**

The first-detection time is written once per unique ID and never overwritten, so the database always preserves when each unit was first seen.

Test Sequence (per motor)

Before power-on — Human visual check (whole shelf at once)

- Connector condition, no visible damage, serial label readable, motor fully seated

Two-stage model: collect, then evaluate

Nothing is computed or judged while testing. The system works in two separable stages:

- **Stage A — Data collection only.** The phases below run on the hardware and **store all raw collected data** into the database — including the large datasets (e.g. Phase 6's dense position samples, the hall waveforms, the temperature series). Large arrays are stored as **binary blobs** to keep the database compact. **No metrics are computed, no PNGs are generated, and no pass/fail is decided during collection** — the goal is to get the raw data off the hardware as fast as possible and move on.
- **Stage B — Post-processing (triggered later from Tab 2, no hardware).** Three things happen on stored data, on demand:
  1. **Metric derivation + pass/fail evaluation** — the **[Run Evaluation]** button derives each phase's metrics from the raw data and applies the current **criteria** to compute pass/fail for every motor and phase. Re-runnable whenever criteria change.
  2. **PNG generation** — a **separate [Generate PNGs]** button renders the plots from the raw data into PNG files (independent of evaluation; can be regenerated on its own).
  3. Both read raw `phase_data` and never alter it, so they can be re-run any number of times.

This means the same raw data can be scored many times under different criteria, and the "good values" are chosen empirically from the distributions rather than guessed before testing. Each phase detail below states what raw data it **collects (Stage A)** and what is checked against criteria in **pass/fail (Stage B)**.

Rules during data collection (Stage A): - **System reset between phases.** A `system_reset` (cmd 27) is issued to each motor before it enters the next phase — the only reliable way to clear a latched fatal error, so a fault in one phase never poisons the next. **Every `system_reset` is followed by a 1-second delay** to let the device come out of the bootloader before any further command is sent (a command arriving during that window would pin the device in the bootloader). This 1 s wait applies to every `system_reset` anywhere in the system, including the [System Reset] button. - **Collection never stops on a bad reading.** A motor that faults or produces an out-of-range value is still parked/handled as the hardware requires (e.g. a burn-in fatal error parks that unit), but the run continues and the raw data is recorded for all units in one pass. - **All raw data is always stored** — whether or not it will later evaluate to a fail. The only gap is a phase that never produced a reading (e.g. the motor went unresponsive), recorded as a missing observation.

The phases below are the **single source of truth** — each is self-contained: what it verifies, how it runs (power / parallelism), its procedure, what it **collects (Stage A)**, its **pass/fail criteria (Stage B)**, and its **configurable parameters**. Truly global settings (serial-port assignment, per-phase enable/disable, run scope) are in *Global Settings*. Disabled phases are skipped; the system-reset-between-phases rule applies across whichever remain enabled.

Phase 1 — Firmware

- **Verifies**: MCU flash + RS485. **Power**: none. **Parallelism**: flash broadcast to all 48 at once; version verify sequential per device.
- Broadcast-flash all 48 motors at once to the target release, then read the firmware version from each device individually (by unique ID).
- **Collected (Stage A)**: the firmware version read back from each device, and whether the device responded after flashing.
- **Pass/fail (Stage B)**: version equals the target release and the device responded.
- **Configurable parameters**: target firmware version (default: latest release, 0.15.0.0).

Phase 2 — Identity

- **Verifies**: MCU, RS485. **Power**: none. **Parallelism**: yes — fast queries.
- Read product info (cmd 22) from each device.
- **Collected (Stage A)**: unique ID, product type, hardware version, software-compatibility code (scc).
- **Pass/fail (Stage B)**: product type / hw version / scc match the expected build.
- **Configurable parameters**: expected product type / hw version / scc (default M17 / 1.5 / 3).

Phase 3 — Electrical baseline

- **Verifies**: voltage divider, thermistor, MCU. **Power**: none. **Parallelism**: yes.
- Read supply voltage (cmd 38), temperature (cmd 42) and status (cmd 16). (The thermal-rise check has its own fresh baseline taken in Phase 11, so this reading is just a sanity check of the at-rest temperature.)
- **Collected (Stage A)**: supply voltage, board temperature, status flags.
- **Pass/fail (Stage B)**: voltage within nominal ± tolerance; temperature within the acceptance range; status flags clean (no fatal error).
- **Configurable parameters**: supply voltage nominal & tolerance (default 24 V ±10%); temperature acceptance range (default 10–45 °C).

Phase 4 — Comms quality

- **Verifies**: RS485 transceiver, MCU UART. **Power**: none. **Parallelism**: yes.
- Send pings (cmd 31) and read the communication statistics (cmd 47).
- **Collected (Stage A)**: ping success count out of the ping total; CRC/error/timeout counters.
- **Pass/fail (Stage B)**: success rate meets the required rate; CRC error count ≤ the max.
- **Configurable parameters**: ping count (default 1000); required success rate (default exactly 100%); max CRC errors (default 0).

Phase 5 — Calibration

- **Verifies**: full motion chain. **Power**: full — high current. **Parallelism**: max 8 per section at once. **Must precede any hall-position read.**
- A batch of 8 motors is calibrated together (respecting the 8-motor power limit).
- **Send `start_calibration` (cmd 6) to all 8 motors in rapid succession**, then **keep the bus completely quiet** for a configurable hold time — **default 30 s** (matches the `-c` calibration flow in `detect_and_set_alias_all_devices.py`, which sends `start_calibration()` then `time.sleep(30)` before reading status).
- The quiet bus is deliberate: **no polling or other RS485 traffic during calibration**, so bus activity can't disturb the measurement. Staying quiet also avoids poking a motor during its post-calibration auto-reboot window (which would otherwise pin it in the bootloader — see the calibration-auto-reboots note).
- **Do not send a `system_reset`** — the motor **resets itself automatically** when calibration finishes. After the quiet hold, **read the status once** (cmd 16) for each motor.
- On a zero status, set `calibration_done` in the database for each motor.
- **Collected (Stage A)**: the single status word read after the quiet hold.
- **Pass/fail (Stage B)**: status is perfectly zero (no fatal error, all flags clear); any non-zero/fatal status fails.
- **Configurable parameters**: calibration bus-quiet hold time (default 30 s).

Phase 6 — Continuous-spin tracking

- **Verifies**: hall sensors, AT5833, motor windings (dynamic tracking). **Power**: full. **Parallelism**: one motor at a time. **Prerequisite**: `calibration_done` set.
- Open-loop mode, maximum motor current.
- **One motor at a time** (not in batches): to gather the maximum number of data points, the host talks to a single motor and polls it as fast as the bus allows. Batching would split the polling bandwidth and reduce point density.
- Command a slow continuous spin (constant low velocity) forward through 1.5 rotations, then reverse back through 1.5 rotations to the start
- While spinning, repeatedly call **`get_comprehensive_position` (cmd 37)** — a single command that returns both the commanded position and the hall sensor position in one round-trip — at the maximum sustainable polling rate, yielding many thousands of points per direction (far more than the 2000-step Phase 7). Using one command instead of two separate reads (cmd 34 + cmd 15) doubles the achievable sample density and guarantees the two positions are captured at the same instant.
- Distinct from Phase 7: this is *continuous motion sampled densely* (dynamic behaviour); Phase 7 is *stepped with settling* (static hysteresis).
- **Collected (Stage A)**: the **entire (commanded, hall) sample stream** for both directions, stored as a **binary blob** in the database (this is a lot of data — it is kept in full, not reduced).
- **Derived in post-processing**: the max absolute hall-vs-commanded deviation (for the criterion); the commanded-vs-hall PNG with forward/reverse traces overlaid.
- **Pass/fail (Stage B)**: max deviation < threshold (default **0.001 rotations**).
- **Configurable parameters**: hall tracking deviation threshold (default 0.001 rot — shared with Phase 7).

Phase 7 — Hall linearity & hysteresis

- **Verifies**: hall sensors, AT5833, motor windings (static hysteresis). **Power**: full. **Parallelism**: yes — 8 at once. **Prerequisite**: `calibration_done` set.
- Open-loop mode, maximum motor current.

- Sweep CW: command 2000 evenly-spaced positions across one full rotation (step size = 1 rotation ÷ 2000); at each step, wait 50 ms for settling, then call **`get_comprehensive_position` (cmd 37)** to read the commanded and hall sensor positions together in one round-trip
- Sweep CCW: repeat the same 2000 positions in reverse order
- Parallelism: fan-out/gather across a group of 8 — command all 8 to step N, wait 50 ms, read all 8, repeat; wall-clock time is the same as testing a single motor.
- **Collected (Stage A)**: the full (commanded, hall) pair at every step for both sweeps (4000 pairs), stored as a binary blob.
- **Derived in post-processing**: the max absolute hall-vs-commanded deviation; the commanded-vs-hall PNG with CW/CCW traces overlaid (hysteresis = the gap between the curves).
- **Pass/fail (Stage B)**: max deviation < threshold (default **0.001 rotations** — shared with Phase 6).
- **Configurable parameters**: hall tracking deviation threshold (default 0.001 rot — shared with Phase 6).

Phase 8 — Hall waveform & peak analysis

- **Verifies**: hall sensors (raw signal quality) and the 50-magnet encoder disk. **Power**: default (per capture program). **Parallelism**: one motor at a time. **Prerequisite**: `calibration_done` set.
- Capture uses the **same capture parameters as `capture_hall_sensor_data.py` defaults**: capture type 1 (raw hall readings), 4000 points, all 3 channels (bitmask 7), 1 time-step per sample, `nSamplesToSum = 64`, `divisionFactor = 16`. The motion, however, spins **1.4 rotations** — the **same distance the firmware uses for calibration** (`CALIBRATION_DATA_COLLECTION_N_TURNS_TIMES_256 = 358`, i.e. 358/256 ≈ 1.4 turns), so the capture yields a bit more than one full turn's worth of extrema (~70 per channel) and we can take the middle 50.
- Each captured value = `sum(64 raw samples) / 16` = **4× a raw hall reading**, so values span roughly 0–65535.
- **Peak/valley finder** mirrors the firmware calibration algorithm (`handle_calibration_logic` in `motor_control.c`): walk the samples tracking a running local max/min and confirm a peak (or valley) once the signal reverses by more than a hysteresis threshold.
  - The firmware uses `HALL_PEAK_FIND_THRESHOLD = 2000` on **raw** readings. Because the captured data is scaled 4× (`nSamplesToSum / divisionFactor = 64/16`), the equivalent threshold here is **2000 × 4 = 8000** (configurable; derived automatically if the capture parameters change).
- **Extrema count check**: the total number of extrema detected per channel must fall within a configurable band — **default 68 … 71** (over 1.4 rotations of a 50-extrema-per-turn disk, ~70 are expected). Too few flags a missing/merged peak (e.g. a dead magnet); too many flags spurious peaks from noise. Outside the band → fail.
- **Extrema selection**: skip the first extremum (as the firmware does — its analysis loop starts at index 1, since the first one forms during acceleration), then take the **middle 50 extrema** (25 peaks + 25 valleys) of the capture for the analysis. 50 = `N_POLES`, where the firmware defines `N_POLES = TOTAL_NUMBER_OF_SEGMENTS / 3 = 150 / 3 = 50` — one per magnet on the 50-magnet disk. Taking the middle 50 (rather than the firmware's last 50) symmetrically drops the acceleration extrema at the start and the deceleration extrema at the end. (The 68–71 count band guarantees there are always at least 50 to select from.)
- **Collected (Stage A)**: the **raw 3-channel hall waveform** (4000 points × 3 channels) stored as a **binary blob** — the full capture is kept, nothing reduced. (The peak-finding, extrema selection and metrics described above and below are all done later in post-processing, not during collection.)
- **Derived in post-processing** — per channel over the selected 25 peaks + 25 valleys:
  - **min and max hall value** per channel
  - **peak height** range — max and min of the confirmed peak/valley amplitudes
  - **peak spacing** range — max and min distance (in samples) between adjacent extrema
  - **max adjacent-peak deviation** — largest y-axis difference between consecutive peaks (peak amplitude irregularity)
  - **max adjacent-valley deviation** — largest y-axis difference between consecutive valleys (valley amplitude irregularity)
  - **average peak height** and **average valley height** (y-axis), and the **span** = average peak – average valley
  - **max peak-from-average deviation** and **max valley-from-average deviation** — the largest distance of any single peak (valley) from the average peak (valley) height
- **Pass/fail (Stage B)** (all criteria configurable):
  - the detected **extrema count** per channel must be within the band — **default 68 … 71**.
  - each channel's min and max hall values must stay inside a band — **default min 1000, max 65535 – 1000 = 64535**. A value at/below 1000 or at/above 64535 indicates the sensor is clipping/saturating against the rail.
  - the **span** (average peak – average valley) must exceed a threshold — **default 40000** — ensuring the hall signal has enough amplitude.
  - every peak must lie within a **max deviation from the average peak — default 2000**; likewise every valley within the same max deviation of the average valley. **Real purpose**: the magnetic encoder disk has **50 magnets**, and each peak/valley in the hall waveform corresponds to one magnet passing the sensor. A single peak or valley that deviates from the average flags an individual magnet that is **damaged or weaker than expected** — this check exists specifically to catch a bad magnet on the disk.
  - the **max adjacent-peak deviation** and the **max adjacent-valley deviation** must each stay below a threshold — **default 2000, configurable** — catching an abrupt amplitude jump between neighbouring magnets.
  - the **peak spacing** (samples between adjacent extrema) must stay within a configurable range — **not too small and not too large** — so both the min and max spacing fall inside the band. Uneven spacing indicates a magnet position error or a missed/spurious extremum. (Defaults to be set from real captures in Tab 2.)
- **PNG (post-processing)**: one PNG per motor with **4 plots, the same layout `capture_hall_sensor_data.py` produces** — hall channels 1, 2, 3 each on its own subplot, plus a 4th with all three overlaid — and **crosses marking the detected peaks and valleys** on each.
- **Configurable parameters**: peak-find hysteresis (default 8000 = firmware 2000 × 64/16); extrema count range (default 68 … 71); saturation band (default 1000 … 64535); span minimum (default 40000); peak/valley max deviation from average (default 2000); max adjacent peak/valley deviation (default 2000); peak-spacing range (min … max, set from real captures).

Phase 9 — Current control

- **Verifies**: current-control path (AT5833 / cmd 28). **Power**: low. **Parallelism**: yes — 8 at once. **Prerequisite**: `calibration_done` set (a hall read decides whether the motor moved).
- Set the **maximum motor current to a low value** via cmd 28 (default 5, in the motor's current units).
- Enable MOSFETs, zero the position, then command a **one-rotation** move; read the hall sensor position (cmd 15) after the move settles.
- This proves current control can be turned down far enough that the motor becomes too weak to spin — a direct functional check of the current-limiting path. The between-phase `system_reset` restores the normal current limit before later phases.
- **Collected (Stage A)**: the hall position change after the one-rotation command at low current.
- **Pass/fail (Stage B, inverted)**: passes if the motor did **NOT** rotate — hall position moved **less than** the no-rotation threshold (default 0.1 rotations). A motor that *does* reach the commanded rotation **fails** (current limit didn't weaken it).
- **Configurable parameters**: low current (default 5); no-rotation threshold (default 0.1 rot).

Phase 10 — Overvoltage protection

- **Verifies**: overvoltage comparator + threshold PWM. **Power**: none (no motion). **Parallelism**: yes — 8 at once.
- ⚠ **Requires new firmware support** — two new test modes must be added to the firmware (via cmd 36) that set the overvoltage-protection threshold to a fixed voltage:
  - one test mode sets the OV threshold to **22 V**
  - the other sets it to **26 V**
  - (On M17 the threshold is set by the overvoltage-setting PWM on PA11 feeding the comparator; the test modes drive that PWM to the two target voltages.)
- With the rack on a **24 V** supply:
  - the **22 V** mode should **trip** overvoltage protection (24 V > 22 V)
  - the **26 V** mode should **not trip** (24 V < 26 V)
- Procedure per motor: enter the 22 V test mode → wait → read status (cmd 16) for the trip; `system_reset`; enter the 26 V test mode → wait → read status; `system_reset`. No motor motion is involved.
- **Firmware dependency**: until these two test modes exist in the firmware, this phase is defined but cannot run — it should be left disabled (unchecked) until the firmware is updated. See *Firmware Dependencies* below.
- **Collected (Stage A)**: the trip result at the 22 V setting and at the 26 V setting (tripped: yes/no for each).
- **Pass/fail (Stage B)**: the 22 V setting tripped **AND** the 26 V setting did not trip. Either wrong → fail.
- **Configurable parameters**: the two threshold setpoints (default 22 V and 26 V) if the firmware test modes expose them; otherwise none.

Phase 11 — Thermal

- **Verifies**: thermistor / thermal behaviour under load. **Power**: full. **Parallelism**: 8 per section at a time.
- **Take a fresh baseline at the start of this phase** (read temperature before running the motor) — Phase 11 does not rely on the Phase 3 baseline.
- Run the motor at full power (8 per section at a time) while **logging temperature (cmd 42) every 1 second** for a configured duration.
- **Collected (Stage A)**: the fresh baseline temperature and the **full per-second temperature series**, stored as a binary blob.
- **Derived in post-processing**: the temperature rise (final – fresh baseline); the best-fit **slope**, **start temperature** (intercept at t = 0) and **R value**; the temperature-vs-time PNG plot.
- **Pass/fail (Stage B)** — all four must hold (all ranges configurable):
  - temperature rise within range (**default 5–20 °C**);
  - best-fit **start temperature** within a configurable range;
  - best-fit **slope** within a configurable range;
  - best-fit **R value** greater than a configurable threshold (a poor fit indicates erratic temperature behaviour).
- **Configurable parameters**: thermal phase duration; rise range (default 5–20 °C); start-temperature range; slope range; R-value minimum.

Phase 12 — Open-loop burn-in

- **Verifies**: full motion chain under sustained load. **Power**: full. **Parallelism**: random 8 per section at any instant. **Duration**: configurable, default 3.5 h.
- **Random-batch scheme**: repeatedly pick **8 motors at random** from the section and spin each a **random distance in a random direction** for a fixed **interval (default 5 s, configurable)**. When the interval ends, pick another random 8 and repeat. Continue for the full configured duration (default 3.5 h).
- Exactly 8 motors are under full current at any instant, respecting the 8-motor power limit.
- Per-motor setup before its first spin: enable → settle ~0.3 s → zero → set tight deviation limit → move (this order avoids spurious deviation trips from the enable transient).
- **Tight position-deviation tolerance** set via cmd 44 (default **0.01 rotations, configurable**). In open loop this catches skipped steps / stalls: if commanded-vs-hall deviation exceeds the limit, the firmware raises a fatal error.
- **Any error during the phase = fail.** A fatal error (deviation trip, driver fault, etc.) on a motor marks its open-loop burn-in **failed** in the database. That motor is then left in its fatal-error state and excluded from further random selection; the remaining motors keep testing for the full duration.
- Temperature (cmd 42) polled periodically for safety/over-temp monitoring.
- **Collected (Stage A)**: any fatal-error event (with code and time).

- **Pass/fail (Stage B)**: no fatal error / deviation trip occurred during the phase.
- **Configurable parameters**: burn-in duration (default 3.5 h); spin interval (default 5 s); deviation tolerance (default 0.01 rot, applied on-device via cmd 44).

Phase 13 — Closed-loop burn-in

- **Verifies**: closed-loop servo performance under sustained load. **Power**: low. **Parallelism**: all 48 per section simultaneously, staggered starts. **Duration**: configurable, default 3.5 h.
- All 48 motors per section run for the full configured duration simultaneously, in **closed loop**.
- Each motor's start is delayed by a small random offset (0–1 s) so acceleration peaks are spread out and the power supply is not overloaded.
- **Motion pattern**: **many, many moves** over the whole duration. Each move is a **trapezoid move** (cmd 2) to a **random displacement within ±N rotations** (N a random magnitude up to a configurable maximum) over a fixed **move duration (default 5 s, configurable)**.
- **No status polling.** After issuing a trapezoid move, the host **does not communicate with that motor until the move duration + 5 %** has elapsed (a small margin so the move has certainly finished). Only then does it **read the max PID error** (cmd 39) and **queue the next move**. This guarantees the move completed before the PID reading without ever polling status, keeping bus traffic minimal so all 48 motors can be serviced.
- **No maximum position deviation is set** (cmd 44 is *not* used here) — we deliberately do **not** want a fatal-error trip from a deviation limit. The cmd 39 read **resets** the value, so each move's max is captured fresh. **The max PID deviation of every move is saved** — the full per-move series, not just the single largest — so its distribution can be histogrammed later. (If a different fatal error does occur, it is still recorded — see below.)
- This phase, not a separate one, is what verifies closed-loop performance (the old standalone closed-loop phase was removed).
- **Collected (Stage A)**: the **max PID deviation of every move** (cmd 39 read-and-reset after each move) — the complete per-move series, stored as a binary blob; any fatal-error event (with code and time); the temperature series (polled every 60 s).
- **Derived in post-processing**: the overall max PID deviation across the phase (for the criterion); the per-move PID-deviation distribution feeds the Phase 13 histogram in its phase tab.
- **Pass/fail (Stage B)**: the overall max PID position deviation is **below a configurable threshold AND** no fatal error occurred during the phase.
- **Configurable parameters**: burn-in duration (default 3.5 h); trapezoid move duration (default 5 s); post-move quiet margin (default 5 %); max move magnitude (±N rotations); max PID deviation threshold.

Phase 14 — Set factory-default alias

- **Verifies**: MCU settings (alias). **Power**: none. **Parallelism**: broadcast set to all; then read back per device.
- Runs **just before** the LED test, so the unit leaves production with the correct factory alias.
- Procedure:
  1. **Broadcast** the set-alias-to-'X' command (cmd 21 to the broadcast address) so every device on the bus is set in one shot.
  2. **Wait ~1 s (configurable).** Setting the alias writes it to non-volatile memory, which **makes the device automatically reboot** — no command can be sent until it comes back up.
  3. **Read the alias back from each device individually, by unique ID** (cmd 22). Addressing must be by unique ID, not alias — once every device is 'X', the alias is no longer a unique address.
- **Collected (Stage A)**: the alias value read back from each device after the reboot. (Collection stores it; it does not itself decide pass/fail.)
- **Pass/fail (Stage B)**: the recorded alias must equal **'X'**.
- **Configurable parameters**: factory alias (default 'X'); post-set reboot delay (default 1 s).

Phase 15 — LED test (must run last)

- **Verifies**: green + red LEDs. **Power**: LEDs only. **Parallelism**: command all at once, then human walk-through.
- Uses the firmware LED test mode (cmd 36, mode 10–13) to drive the green and red LEDs solid ON
- **This locks the motor up** — the firmware disables interrupts and spins forever in the LED test mode, so the device stops responding on RS485 and only a power cycle recovers it
- Because it bricks the unit until power-cycled, this is the **last** phase — nothing automated can run after it

Procedure: 1. Send the LED-on test-mode command to every motor in the test set (all LEDs go solid on; every motor locks up) 2. Human walks the rack and visually checks that both LEDs are lit on every motor 3. A **popup asks: "Did all motors pass the LED test (all LEDs lit)?"** - **Yes →** the LED test is marked **passed for every motor** in the test set. Human power-cycles the rack to recover the motors. Done. - **No →** the failed-motor reconciliation flow below runs.

Failed-motor reconciliation (when the human answers No): 1. Human **physically removes the failing motor(s)** from the rack 2. Human **power-cycles** the rack — this both recovers the still-present motors from the LED-test lockup and leaves the removed ones absent 3. Human clicks **"Check what motor was removed"** 4. The program **pings every motor in the test set** : motors that respond are still present; motors that do not respond are the ones that were removed 5. The program **lists the missing (removed) motors** to the human for confirmation 6. On confirmation, the LED test is marked **passed** for the motors that responded to the ping, and **failed** for the missing motors

- **Collected (Stage A)**: the human confirmation per device (LEDs lit: yes/no), determined directly or via the removal-and-ping reconciliation
- **Pass/fail (Stage B)**: that confirmation must be "yes"
- **Configurable parameters**: none

- Note: the power cycle must happen **before** the ping check — a motor still in LED-test lockup would not respond and would be mis-flagged as removed

---

Timing Estimate

Stage	Wall-clock time
Phases 1–5 (firmware, identity, electrical, comms quality, calibration)	~30 min
Phase 6: continuous-spin tracking (one motor at a time, dense sampling)	~15 min
Phase 7: hall linearity sweep (2000 steps × 2 directions × 50 ms settle, 8 in parallel)	~10 min
Phase 8: hall waveform & peak analysis (one motor at a time, ~9 s spin each)	~10 min
Phases 9–11 (current control, overvoltage, thermal)	~15 min
Phase 12: open-loop burn-in (configurable, default 3.5 h)	~3.5 h
Phase 13: closed-loop burn-in (configurable, default 3.5 h)	~3.5 h
Phase 14: set factory alias 'X' + record (sequential per device)	~2 min
Phase 15: LED test + human check + power cycle	~10 min
Total (defaults)	**~7.9 h**

All 3 bus sections run in parallel throughout. The bottleneck is the two burn-in phases running back-to-back; their durations are configurable, so the total scales with whatever burn-in times you set. Disabling phases via the checkboxes shortens the run accordingly.

System Architecture

```
Python backend (FastAPI + asyncio)
  ├── Bus worker A ── user-selected port  (shelves 1–2, 48 motors)
  ├── Bus worker B ── user-selected port  (shelves 3–4, 48 motors)
  ├── Bus worker C ── user-selected port  (shelves 5–6, 48 motors)
  ├── Serial-port assignment — each bus is bound to a port chosen in the UI;
  |     the backend also rejects any assignment with a duplicate port (defence
  |     in depth beyond the UI greying-out), and lists available ports on request
  ├── SQLite database (WAL mode, shared across workers)
  ├── Persistent settings file (JSON on disk) — all config-panel values AND the
  |     serial-port assignment are saved on every change and reloaded at startup,
  |     so they survive restarts. Writes are ATOMIC: write to a temp file, fsync,
  |     then os.replace() over the old file — a crash mid-write never corrupts
  |     or loses the settings
  └── WebSocket broadcast → live rack state to all connected browsers

All test execution runs in the BACKEND. The bus workers drive the tests
and write results to the database on their own; the browser is only a
view/control client. **Closing or reloading the browser does not affect a
running test** — there is no test state in the page. On (re)connect the
browser fetches the current state from the backend and resumes showing live
progress; with no browser open at all, tests keep running to completion.
Start/Stop/Pause/Cancel are commands sent to the backend, not loops in the
page, so a dropped connection never stops a run.

Browser UI (plain HTML + JavaScript, no build step). Tabs: **Detect & Collect**,
**Database**, then **one tab per phase** (Phase 1 … Phase 15).

TAB 1 — Detect & Collect (Stage A)
  ├── Serial port assignment — a dropdown per bus (A / B / C) listing the host's
  |     serial ports (/dev/cu.* or /dev/ttyUSB*) + [Refresh ports]. UNIQUENESS
  |     enforced: a port picked for one bus is greyed out in the others; Start is
  |     disabled until all three buses have distinct ports. Persisted.
  ├── Detection panel — one column per bus, each with:
  |     [Detect] button + pending-passes counter (++ on click, -- on finish)
  |     [Clear] button (empties this bus's test set only)
  |     live test-set list + motor count
  |     list colour: Green=new/known-in-set  Orange=in DB but not in set
  ├── [Clear All] — empties all three test sets (DB untouched)
  ├── Run scope: [all devices in test set] / [only devices with incomplete data]
  ├── 6 × 24 rack grid — LIVE collection status (position = bus + discovery
```

```
  |     order, not persisted): Gray=no data  Yellow=collecting  Blue=collected
  |     Orange=awaiting human confirm
  ├──── LED-test popup (after Phase 15): "Did all motors pass?" Yes → record all
  |     as pass; No → [Check what motor was removed] pings the set, lists
  |     missing motors, records present=pass / missing=fail for the LED check
  ├──── Controls: Start All / Start Section / Stop / Pause / Re-run phase
  ├──── [Cancel] — aborts any running operation, leaving motors where they stopped
  └──── [System Reset] — broadcasts system_reset (cmd 27) to all motors on all
        buses; waits 1 s afterward to let devices exit the bootloader

TAB 2 — Database (filter + browse + results)
  ├──── Filter bar — unique ID, date range, firmware, product/hw/scc, by phase
  |     result (e.g. failed Phase 8), pass/fail, cleared-or-not
  ├──── Scope selector: [only devices in the test set] / [all devices in the DB]
  |     — THIS filtered set is what feeds the histograms in every phase tab
  ├──── Device table — one row per device: overall pass/fail (from the latest
  |     evaluation) and, for fails, the failing phase(s) + metric/value; links
  |     to the device's full data and plots
  ├──── [Identify] on every row — sends Identify (cmd 41) so the device flashes
  |     its LEDs and is easy to locate in the rack
  ├──── Per-device "Clear failure" → operator override (name + note)
  ├──── [Run Evaluation] — post-processing: derives every phase's metrics from
  |     the raw phase_data and applies all current criteria, writing fresh
  |     phase_eval rows (no hardware; re-runnable; evaluates ALL criteria)
  ├──── [Generate PNGs] — SEPARATE post-processing step: renders all plots from
  |     the raw phase_data into PNG files on disk (independent of evaluation;
  |     can be regenerated on its own). Each PNG filename includes the unique ID.
  ├──── [Show PNGs] — displays the generated PNGs inline on this page, GROUPED BY
  |     PLOT TYPE: first all hall-sensor plots (Phases 6/7/8), then temperature
  |     plots (Phase 11 / burn-ins), etc. — so you can scroll and eyeball
  |     similar plots together. Each plot type has its own show/hide toggle, and
  |     every PNG is labelled with its device's unique ID. Honours the filter
  |     above (only the filtered devices' PNGs are shown).
  ├──── Summary — total / pass / fail / yield % under the current criteria
  └──── Export filtered set (CSV + PNG)

TAB 3 … 17 — one tab PER PHASE (Phase 1 … Phase 15)
  Each phase tab is self-contained and contains:
  ├──── Phase description — what this phase does
  ├──── Enable/disable toggle for the phase
  ├──── Input (collection) parameters — editable (e.g. Phase 5 bus-quiet hold;
  |     Phase 8 peak-find hysteresis; Phase 11 duration; burn-in durations)
  └──── One block PER MEASURED ITEM, with the criterion and its histogram side by
        side:
          • the pass/fail criterion parameter(s) for that item, editable
          • the histogram of that item, built from the Tab 2 FILTERED set, with
            VERTICAL LINES drawn at the current threshold(s)
        (Phases whose observation is categorical/boolean — e.g. firmware
         version, alias, overvoltage trips, LED yes/no — show a count/bar
         summary instead of a histogram, with the criterion beside it.)

Changing a criterion in a phase tab and pressing [Run Evaluation] (Tab 2)
re-scores pass/fail from the stored data — no hardware re-test.

No authentication — open access on the local network.
```

Database

Three tables in a single SQLite file, keyed entirely on the permanent hardware unique ID (no slot is recorded):

```
motors        unique_id (PK), product_type, hw_version, scc,
              calibration_done (BOOL), first_detected, last_seen

phase_data    data_id (PK), unique_id (FK), phase, collected_at,
              firmware_version, raw_blob (BLOB), scalar, observation, detail
                        -- Stage A: ALL raw data; large arrays in raw_blob

phase_eval    eval_id (PK), unique_id (FK), phase, evaluated_at,
              criteria_version, derived_metrics, result (pass/fail),
              failing_metric, cleared (BOOL), cleared_by, cleared_note
                        -- Stage B: derived metrics + computed pass/fail
```

The split mirrors the two-stage model: **`phase_data`** holds **all raw data collected on the hardware** (Stage A) and is never overwritten — large arrays (Phase 6/7 sample streams, Phase 8 waveforms, the temperature series, the per-move PID series) go in `raw_blob` as **binary** to keep the file compact; small scalars/events (supply voltage, overvoltage trips, LED yes/no, calibration status) go in `scalar` / `observation`. **`phase_eval`** holds the

derived metrics (computed from the raw data during post-processing) and the pass/fail. The per-phase tabs render histograms from these derived metrics over the Tab 2 filtered set.

PNG files are NOT stored in the database. They are generated in post-processing into a dedicated folder on disk, each **filename containing the device's unique ID** (and the plot type), e.g. `plots/hall_waveform_<UNIQUEID>.png`. The DB does not reference them — the viewer finds them in the folder by unique ID and plot type.

PNGs are fully disposable. They are pure derived artifacts of the raw data in `phase_data`, so deleting the plots folder loses nothing — **[Generate PNGs]** rebuilds every PNG from the database. (The same is true of all derived metrics: they come from the raw data, so the database alone is the authoritative record.)

How current pass/fail is derived: running **Stage B evaluation** inserts a fresh `phase_eval` row per (motor, phase) using the current criteria. The *latest* eval row per (motor, phase) is authoritative. So: - **Re-evaluate after changing criteria** — re-running Stage B writes new eval rows from the existing `phase_data`; **no hardware re-test needed**. The collected data is untouched. - **Re-collect** — only needed if you want fresh measurements; inserts new `phase_data`, then Stage B is re-run. - **Manual clear** — an operator can set `cleared = TRUE` (with name and note) on a failing eval row, for issues fixed outside the data. - A motor's overall result = pass only if every enabled phase's latest eval row is pass-or-cleared.

Every motor accumulates full history under its unique ID. The DB file holds all raw data (with large arrays as binary blobs); the generated PNGs live in a separate plots folder named by unique ID. Both are portable — copy them for off-site records.

Database tab (Tab 2)

The tab for browsing the database and viewing pass/fail results. It does **not** edit criteria (that happens in the per-phase tabs) — it filters the data and shows which devices pass or fail under the current criteria.

- **Filter bar** (filters combine with AND): unique ID; date range; firmware version; product / hw version / scc; result on a specific phase (e.g. "failed Phase 8"); overall pass/fail; include/exclude cleared failures.
- **Scope selector**: **only devices in the current test set** (what's physically on the rack) or **all devices in the database** (full history). **This filtered set is what every phase tab's histograms are built from** — narrow it here to analyse any slice (a date range, one firmware version, only failures, etc.). Measurements from failed phases are included, so the distributions reflect the full population.
- **Device table** — one row per device: overall result (from the latest evaluation) and, for a fail, the failing phase(s) plus the specific metric and value outside its limit. Each row links to that device's full collected data and plots; sortable columns.
- **[Identify] on every row** — sends Identify (cmd 41) so the device flashes its LEDs, making the exact unit easy to find in the rack (especially the failed ones to pull). Works on any device currently on the bus.
- **Operator override** — a unit failing for a reason resolved outside the data can be cleared (with name + note); the override survives re-evaluation until that phase's criteria change.
- **[Run Evaluation]** — applies all current criteria (from every phase tab) to the stored `phase_data` and writes fresh `phase_eval` rows. No hardware; safe to run any time. Because it re-scores from stored data, changing a criterion and re-evaluating immediately shows which units flip.
- **Summary** — total / pass / fail / yield % under the current criteria; can be exported (CSV + PNG).

Per-phase tabs (Tab 3 … 17)

There is **one tab per phase** (Phase 1 … Phase 15). Each is the single place that gathers everything about that phase, so it can be understood and tuned on its own:

- **Description** — what the phase does (mirrors the phase's section above).
- **Enable/disable toggle** and the phase's **input (collection) parameters**, editable.
- **One block per measured item**, with the **criterion and its histogram in close proximity**:
  - the editable **pass/fail criterion parameter(s)** for that item;
  - immediately beside it, the **histogram** of that item across the **Tab 2 filtered set**, with **vertical lines at the current threshold(s)** — so you can see exactly where a limit sits in the real distribution and how many units it would pass or fail.
- Phases whose observation is **categorical/boolean** (firmware version, identity match, calibration status, overvoltage trips, alias, LED yes/no) show a **count or bar summary** beside the criterion instead of a histogram.

The histograms plot the **derived metrics**, which are produced by post-processing — so they populate after **[Run Evaluation]** has been run at least once on the collected data (re-running it after a criterion change refreshes them).

This is where criteria get tuned empirically: collect a batch (Tab 1, no pass/fail yet), open the relevant phase tab, drag the threshold to cleanly separate the good population from the outliers in its histogram, then hit **[Run Evaluation]** (Tab 2) to re-score from the stored data — no hardware re-test — and see the result in the Tab 2 device table. The defaults in this document are starting points to be replaced with production-tuned values this way. Each criteria change saves a **criteria version**, recorded with every evaluation.

Global Settings

Per-phase parameters live in each phase's section above (each phase lists its own **Configurable parameters**). Only the settings that are not

specific to a single phase are listed here. All settings — global and per-phase — are **persisted to a JSON file on disk** (atomic write: temp file → fsync → `os.replace()`), saved on every change and reloaded at startup, so they survive restarts.

Setting	Default	Notes
Serial port per bus (A/B/C)	none	One serial port assigned to each bus; must be three distinct ports (enforced in UI and backend)
Phase enable/disable (1–15)	all enabled	Per-phase checkbox; disabled phases are skipped
Run scope	all devices in test set	Run on all devices in the test set, or only on devices with **incomplete data** (missing a collected result for one or more enabled phases)

Firmware Dependencies

Most phases work with the current firmware (0.15.0.0). One phase needs new firmware before it can run:

- **Phase 10 — Overvoltage protection** requires **two new test modes** (cmd 36) that set the overvoltage-protection threshold to fixed voltages: one to **22 V** and one to **26 V** (by driving the OV-threshold PWM on PA11). Until these exist, leave Phase 10 disabled (its enable toggle in the per-phase configuration). All other phases are unaffected.

Decisions (resolved)

1. **Firmware**: always flash the latest release (currently 0.15.0.0).
2. **Slot tracking**: not recorded — the unique ID is the sole identifier. The rack grid is a live view only.
3. **Collect then evaluate**: the rack only collects data (system-reset between every phase, never stop early). Pass/fail is computed separately in Stage B by applying criteria to the stored data, and can be re-run whenever criteria change. Failures can be cleared by re-evaluation or an operator override.
4. **Authentication**: none — open on the local network.